Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(Empowered Autonomous Institution under R.T.M Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++ Grade
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of CSE-IoT

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

Session 2025-2026(EVEN)

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

SHEIKH WASIMUDDIN
Name and Signature of Student and Date
(Signature and Date in Handwritten)

Sheikh

21/03/26

Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(Empowered Autonomous Institution under R.T.M Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++ Grade
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of CSE-IoT

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

Session	**2024-25 (EVEN)**	**Course Name**	**Mobile and Web Application for IOT Lab**
Semester	**6**	**Course Code**	**23IOT1624**
Roll No	**63**	**Name of Student**	**SHEIKH WASIMUDDIN**

Practical Number	09
Course Outcome	1. Understand and apply basic Kotlin programming constructs including variables, control structures, functions, null safety, and object-oriented concepts for Android app development. 2. Design and develop responsive user interfaces using XML layout files, ViewGroups, and widgets in Android Studio, adhering to Material Design principles. 3. Implement event handling and application logic using Kotlin to capture user input, manage app state, navigate between screens, and interact with Android system components like Toasts and Dialogs 4. Integrate learned concepts to develop a mini Android application that demonstrates use of ViewModel, multi-screen navigation, and proper UI/UX design for real-world problem-solving
Aim	Smart Multi-Utility Unit Converter.
Problem Definition	In everyday life, people often need to convert values between different units such as length, weight, temperature, and more. Manually calculating these conversions can be time-consuming and prone to errors. There is a need for a simple, fast, and user-friendly mobile application that allows users to perform accurate unit conversions instantly in one place.
Theory (100 words)	Unit conversion is an essential concept used in various fields such as science, engineering, and daily activities. It involves transforming a value from one measurement unit to another using standard formulas or conversion factors. In mobile applications, implementing unit conversion requires handling user inputs, applying correct mathematical logic, and displaying results instantly. Kotlin, being a modern programming language for Android development, provides efficient tools for building such applications with clean syntax and better performance. By integrating UI components with logical operations, the application ensures accurate and user-friendly conversion of different units.

Procedure and Execution (100 Words)	**Step for Implementation:** 1. Designed the user interface using XML layouts 2. Created multiple categories for different unit types 3. Used Kotlin to handle user input and logic 4. Implemented conversion formulas for each unit 5. Connected UI elements with backend logic using listeners 6. Displayed real-time results based on user input 7. Tested the app for accuracy and performance 8. Improved UI for better user experience 9. Debugged errors and optimized performance 10. Finalized the application for smooth functionality

Code:

1. **AreaConverterController.java**

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.AreaConverterActivity;
import com.mk.model.AreaConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class AreaConverterController {
   private final AreaConverterModel model;
   private final AreaConverterActivity view;

   public AreaConverterController(AreaConverterModel model,
AreaConverterActivity view) {
      this.model = model;
      this.view = view;
   }
```

```
    public double performConversion(TextView selectedValue){
        String value = selectedValue.getText().toString();
        if (value.isEmpty()) {
            return 0.0;
        }

        double resultValue = 0.0;

        try {
            if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
                value = value.replace("×", "*").replace("÷", "/");
                //Log.w("AreaApp", "Before Expression: " + value);
                if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
                    value = value.substring(0,value.length()-1);
                }
                //Log.w("AreaApp", "After Expression: " + value);
                resultValue = ExpressionEvaluator.evaluateExpression(value);
            } else {
                resultValue = Double.parseDouble(value);
            }
        } catch (Exception e) {
            Log.e("AreaApp", "Error parsing value: " + e.getMessage(), e);
            return Double.NaN;
        }

        int idx = view.values.indexOf(selectedValue);
        TextView baseUnitTextView = (TextView) view.units.get(idx);
        String baseUnit = baseUnitTextView.getText().toString();
        int targetIdx = (idx + 1) % 2;
        TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
        String targetUnit = targetUnitTextView.getText().toString();

        double conversionFactor =
Objects.requireNonNull(model.getAreaUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getAreaUnitsWithFactors().get(targetUnit));
        resultValue *= conversionFactor;

        return resultValue;
    }
}
```

2. CurrencyConverterController.java

```
package com.mk.controller;


import com.mk.model.CurrencyConverterModel;
import com.mk.convert.CurrencyConverterActivity;

import java.math.BigDecimal;
import java.util.List;

public class CurrencyConverterController {
   private final CurrencyConverterModel model;
   private final CurrencyConverterActivity view;

   public CurrencyConverterController(CurrencyConverterActivity view,
CurrencyConverterModel model) {
      this.view = view;
      this.model = model;

   }
   public void handleButtonClick(double amount,String fromCurrency, String
toCurrency1, String toCurrency2){
      List<BigDecimal> convertedAmounts =
model.convertCurrency(BigDecimal.valueOf(amount), fromCurrency,
toCurrency1, toCurrency2);

view.updateUI(convertedAmounts.get(0).doubleValue(),convertedAmounts.get(1
).doubleValue());
   }
}
```

3. DataConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.DataConverterActivity;
import com.mk.model.DataConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class DataConverterController {
   private final DataConverterModel model;
   private final DataConverterActivity view;

   public DataConverterController(DataConverterModel model,
DataConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

```
//Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("DataApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getDataUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getDataUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

4. EnergyConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.EnergyConverterActivity;
import com.mk.model.EnergyConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class EnergyConverterController {

   private final EnergyConverterModel model;
   private final EnergyConverterActivity view;

   public EnergyConverterController(EnergyConverterModel model,
EnergyConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

```
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("EnergyApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getEnergyUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getEnergyUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

5. ForceConvertorController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.ForceConverterActivity;
import com.mk.model.ForceConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class ForceConverterController {

   private final ForceConverterModel model;
   private final ForceConverterActivity view;

   public ForceConverterController(ForceConverterModel model,
ForceConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

```
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("ForceApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getForceUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getForceUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

6. LengthConvertorController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.LengthConverterActivity;
import com.mk.model.LengthConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class LengthConverterController {
   private final LengthConverterModel model;
   private final LengthConverterActivity view;

   public LengthConverterController(LengthConverterModel model,
LengthConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

```
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("LengthApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getLengthUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getLengthUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

7. MassConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.MassConverterActivity;
import com.mk.model.MassConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class MassConverterController {
   private final MassConverterModel model;
   private final MassConverterActivity view;

   public MassConverterController(MassConverterModel model,
MassConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
```

```
            } else {
                resultValue = Double.parseDouble(value);
            }
        } catch (Exception e) {
            Log.e("MassApp", "Error parsing value: " + e.getMessage(), e);
            return Double.NaN;
        }

        int idx = view.values.indexOf(selectedValue);
        TextView baseUnitTextView = (TextView) view.units.get(idx);
        String baseUnit = baseUnitTextView.getText().toString();
        int targetIdx = (idx + 1) % 2;
        TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
        String targetUnit = targetUnitTextView.getText().toString();

        double conversionFactor =
Objects.requireNonNull(model.getMassUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getMassUnitsWithFactors().get(targetUnit));
        resultValue *= conversionFactor;

        return resultValue;
    }
}
```

8. NumericalConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.NumeralsConverterActivity;
import com.mk.model.NumeralsConverterModel;
import com.mk.utils.ExpressionEvaluator;

public class NumeralsConverterController {

   private final NumeralsConverterModel model;
   private final NumeralsConverterActivity view;

   public NumeralsConverterController(NumeralsConverterModel model,
NumeralsConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public String performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return String.valueOf(0.0);
      }

      Double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
         } else {
            resultValue = Double.parseDouble(value);
```

```
            }
        } catch (Exception e) {
            Log.e("NumeralsApp", "Error parsing value: " + e.getMessage(), e);
            return String.valueOf(Double.NaN);
        }

        int idx = view.values.indexOf(selectedValue);
        TextView baseUnitTextView = (TextView) view.units.get(idx);
        String baseUnit = baseUnitTextView.getText().toString();
        int targetIdx = (idx + 1) % 2;
        TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
        String targetUnit = targetUnitTextView.getText().toString();


        return
convertNumeral(String.valueOf(resultValue.intValue()),baseUnit,targetUnit);
    }
    public String convertNumeral(String value, String fromUnit, String toUnit) {
        // Get the bases for the source and target numeral systems
        int fromBase =
NumeralsConverterModel.numeralUnitsWithBases.getOrDefault(fromUnit, -1);
        int toBase =
NumeralsConverterModel.numeralUnitsWithBases.getOrDefault(toUnit, -1);

        if (fromBase == -1 || toBase == -1) {
            throw new IllegalArgumentException("Invalid numeral system: " +
fromUnit + " or " + toUnit);
        }

        // Convert the value to an integer (base 10) from the source base
        int decimalValue = Integer.parseInt(value, fromBase);

        // Convert the decimal value to the target base
        return Integer.toString(decimalValue, toBase).toUpperCase(); // Uppercase
for Hexadecimal representation
    }

}
```

9. PressureConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.PressureConverterActivity;
import com.mk.model.PressureConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class PressureConverterController {

   private final PressureConverterModel model;
   private final PressureConverterActivity view;

   public PressureConverterController(PressureConverterModel model,
PressureConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

```
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("PressureApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getPressureUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getPressureUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

10. SpeedConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.SpeedConverterActivity;
import com.mk.model.SpeedConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class SpeedConverterController {
   private final SpeedConverterModel model;
   private final SpeedConverterActivity view;

   public SpeedConverterController(SpeedConverterModel model,
SpeedConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
```

```
            } else {
                resultValue = Double.parseDouble(value);
            }
        } catch (Exception e) {
            Log.e("SpeedApp", "Error parsing value: " + e.getMessage(), e);
            return Double.NaN;
        }

        int idx = view.values.indexOf(selectedValue);
        TextView baseUnitTextView = (TextView) view.units.get(idx);
        String baseUnit = baseUnitTextView.getText().toString();
        int targetIdx = (idx + 1) % 2;
        TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
        String targetUnit = targetUnitTextView.getText().toString();

        double conversionFactor =
Objects.requireNonNull(model.getSpeedUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getSpeedUnitsWithFactors().get(targetUnit));
        resultValue *= conversionFactor;

        return resultValue;
    }
}
```

11. TemperatureConverterController.java

```
package com.mk.controller;


import static
com.mk.model.TemperatureConverterModel.temperatureUnitsWithFactors;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.TemperatureConverterActivity;
import com.mk.model.TemperatureConverterModel;
import com.mk.utils.ExpressionEvaluator;

public class TemperatureConverterController {
   private final TemperatureConverterModel model;
   private final TemperatureConverterActivity view;

   public TemperatureConverterController(TemperatureConverterModel model,
TemperatureConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if(value.startsWith("-")) {
            value = value.substring(1);
            resultValue = -1 * Double.parseDouble(value);
         } else if (value.contains("-") && value.length() == 1) {
            return Double.NaN;
         } else {
            resultValue = Double.parseDouble(value);
         }
```

```
        } catch (Exception e) {
            Log.e("TemperatureApp", "Error parsing value: " + e.getMessage(), e);
            return Double.NaN;
        }


        int idx = view.values.indexOf(selectedValue);
        TextView baseUnitTextView = (TextView) view.units.get(idx);
        String baseUnit = baseUnitTextView.getText().toString();
        int targetIdx = (idx + 1) % 2;
        TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
        String targetUnit = targetUnitTextView.getText().toString();

        try {
            resultValue = convertTemperature(resultValue, baseUnit, targetUnit);
        }catch (IllegalArgumentException e){
            Log.e("TemperatureApp", "Error converting temperature: " +
e.getMessage(), e);
            return Double.NaN;
        }

        return resultValue;
    }

    public static double convertTemperature(double value, String fromUnit, String
toUnit) {
        if (fromUnit.equals(toUnit)) {
            return value; // No conversion needed
        }

        // Convert to Kelvin as the base unit
        double valueInKelvin = toKelvin(value, fromUnit);

        // Convert from Kelvin to the target unit
        return fromKelvin(valueInKelvin, toUnit);
    }
```

```
    private static double toKelvin(double value, String fromUnit) {
        switch (fromUnit) {
            case "Celsius":
                return value + 273.15;
            case "Fahrenheit":
                return (value + 459.67) * 5 / 9;
            case "Rankine":
                return value * 5 / 9;
            case "Réaumur":
                return value * 1.25 + 273.15;
            case "Kelvin":
                return value; // Already Kelvin
            default:
                throw new IllegalArgumentException("Invalid fromUnit: " +
fromUnit);
        }
    }

    private static double fromKelvin(double value, String toUnit) {
        switch (toUnit) {
            case "Celsius":
                return value - 273.15;
            case "Fahrenheit":
                return value * 9 / 5 - 459.67;
            case "Rankine":
                return value * 9 / 5;
            case "Réaumur":
                return (value - 273.15) * 0.8;
            case "Kelvin":
                return value; // Already Kelvin
            default:
                throw new IllegalArgumentException("Invalid toUnit: " + toUnit);
        }
    }
}
```

12. TimeConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.TimeConverterActivity;
import com.mk.model.TimeConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class TimeConverterController {
   private final TimeConverterModel model;
   private final TimeConverterActivity view;

   public TimeConverterController(TimeConverterModel model,
TimeConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

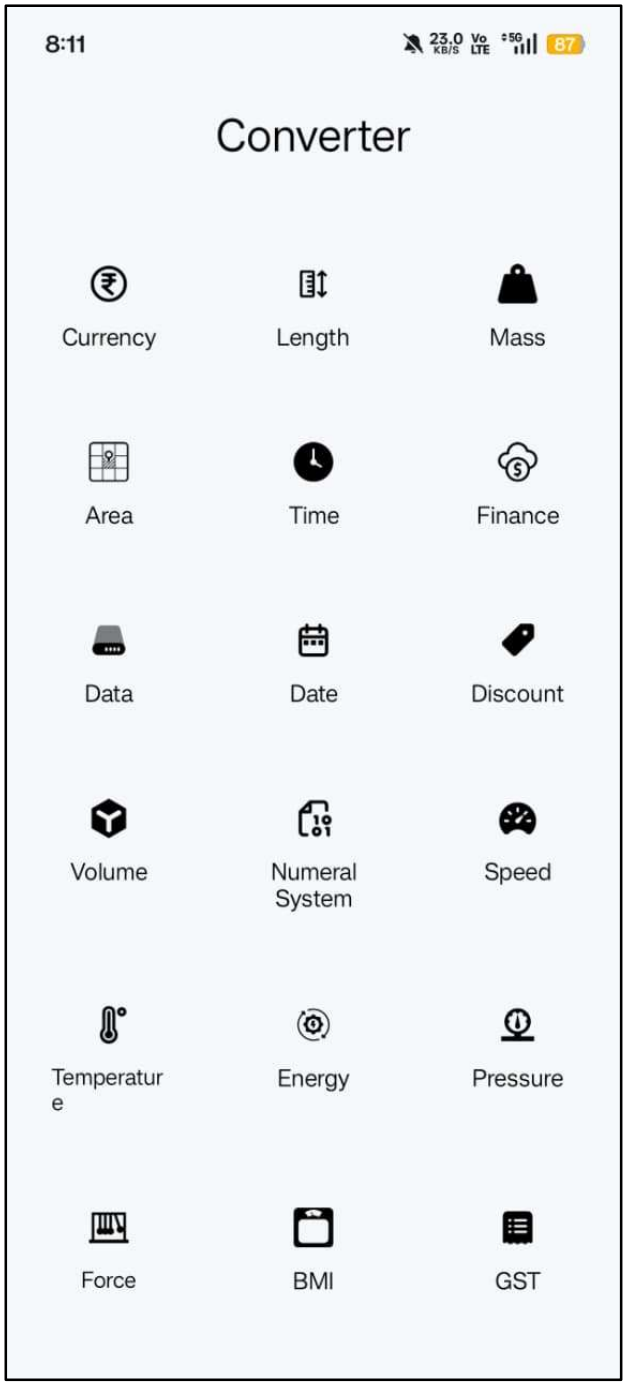
```
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("TimeApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getTimeUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getTimeUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

13. VolumeConverterController.java

```
package com.mk.controller;

import android.util.Log;
import android.widget.TextView;

import com.mk.convert.VolumeConverterActivity;
import com.mk.model.VolumeConverterModel;
import com.mk.utils.ExpressionEvaluator;

import java.util.Objects;

public class VolumeConverterController {
   private final VolumeConverterModel model;
   private final VolumeConverterActivity view;

   public VolumeConverterController(VolumeConverterModel model,
VolumeConverterActivity view) {
      this.model = model;
      this.view = view;
   }

   public double performConversion(TextView selectedValue){
      String value = selectedValue.getText().toString();
      if (value.isEmpty()) {
         return 0.0;
      }

      double resultValue = 0.0;

      try {
         if (value.contains("+") || value.contains("-") || value.contains("×") ||
value.contains("÷")) {
            value = value.replace("×", "*").replace("÷", "/");
            //Log.w("AreaApp", "Before Expression: " + value);
            if(value.endsWith("+") || value.endsWith("-") || value.endsWith("*") ||
value.endsWith("/")){
               value = value.substring(0,value.length()-1);
            }
```

```
            //Log.w("AreaApp", "After Expression: " + value);
            resultValue = ExpressionEvaluator.evaluateExpression(value);
        } else {
            resultValue = Double.parseDouble(value);
        }
    } catch (Exception e) {
        Log.e("VolumeApp", "Error parsing value: " + e.getMessage(), e);
        return Double.NaN;
    }

    int idx = view.values.indexOf(selectedValue);
    TextView baseUnitTextView = (TextView) view.units.get(idx);
    String baseUnit = baseUnitTextView.getText().toString();
    int targetIdx = (idx + 1) % 2;
    TextView targetUnitTextView = (TextView) view.units.get(targetIdx);
    String targetUnit = targetUnitTextView.getText().toString();

    double conversionFactor =
Objects.requireNonNull(model.getVolumeUnitsWithFactors().get(baseUnit)) /

Objects.requireNonNull(model.getVolumeUnitsWithFactors().get(targetUnit));
    resultValue *= conversionFactor;

    return resultValue;
  }
}
```

Output:

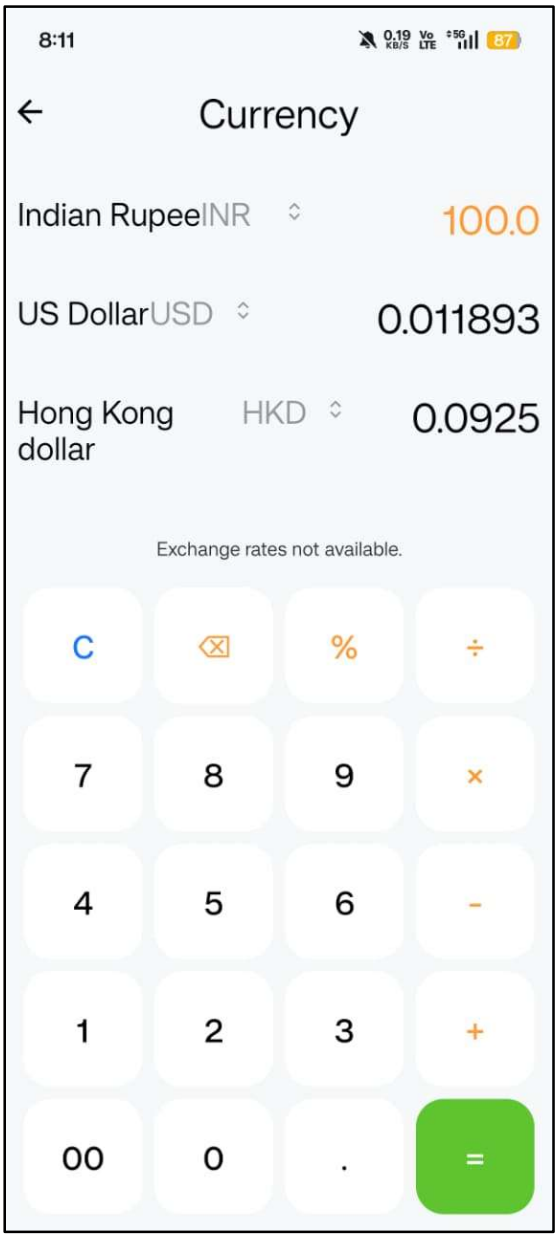

8:11
Converter
Currency
Length
Mass
Area
Time
Finance
Data
Date
Discount
Volume
Numeral System
Speed
Temperature
Energy
Pressure
Force
BMI
GST

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

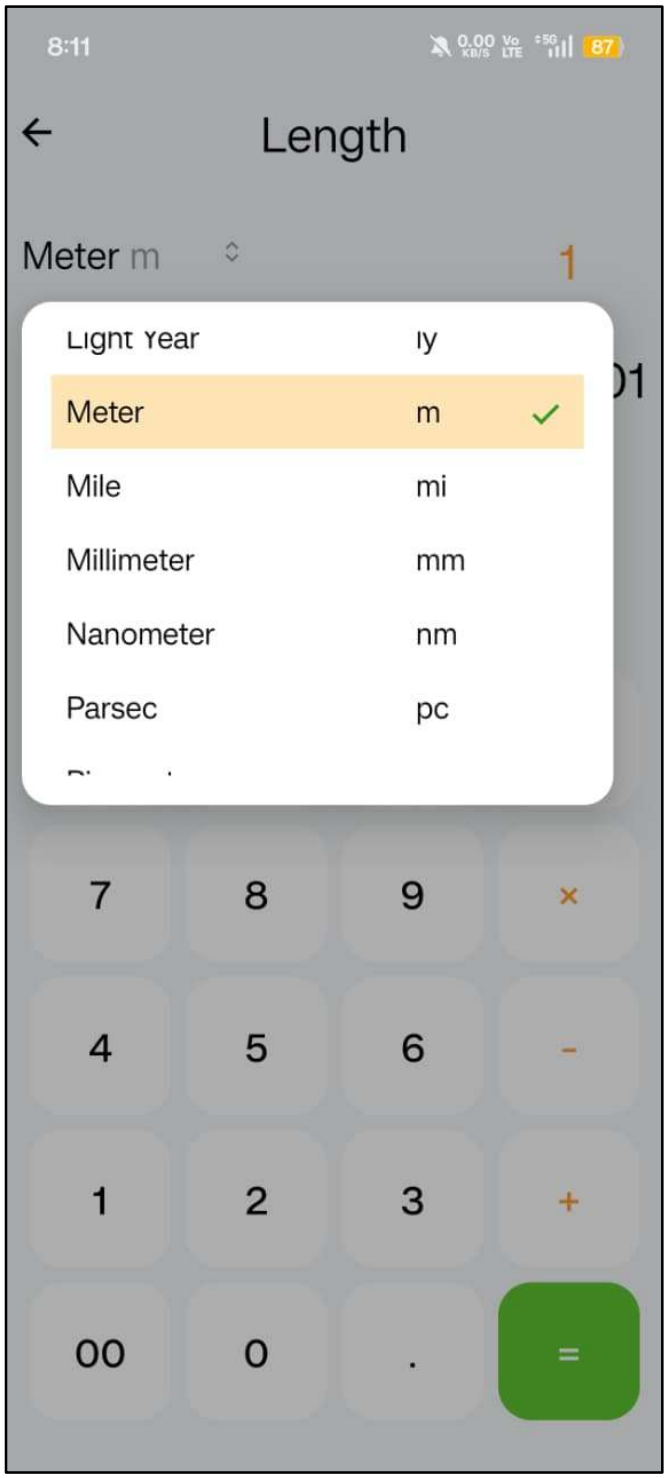


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

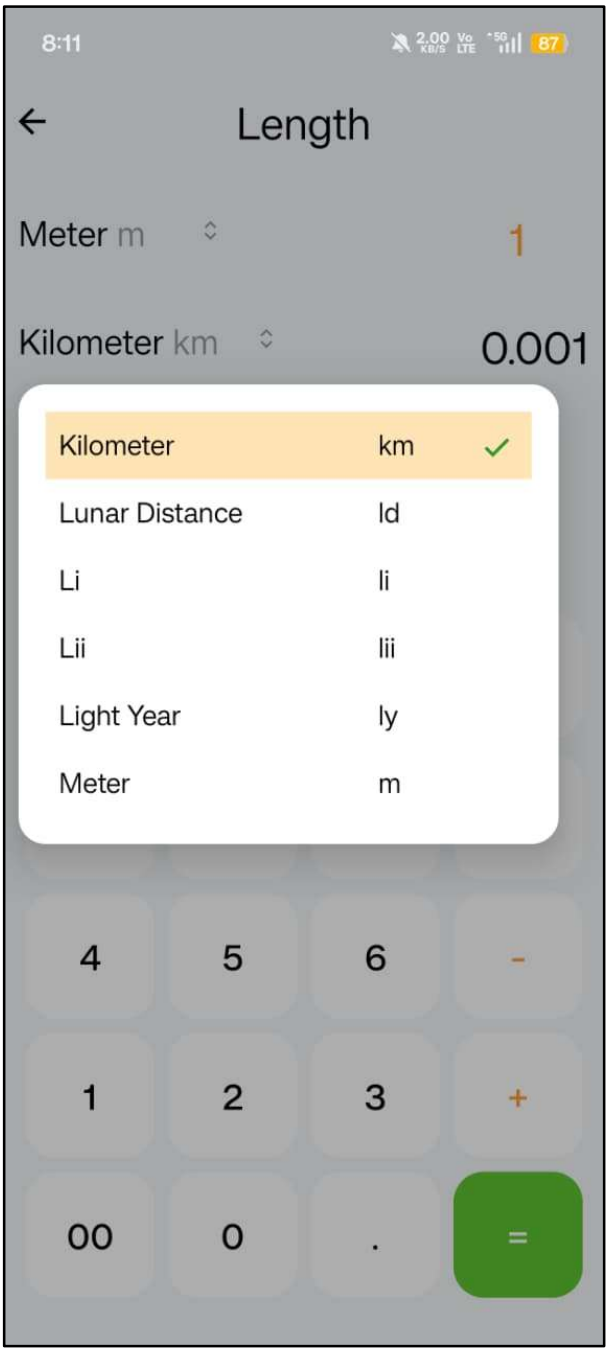


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

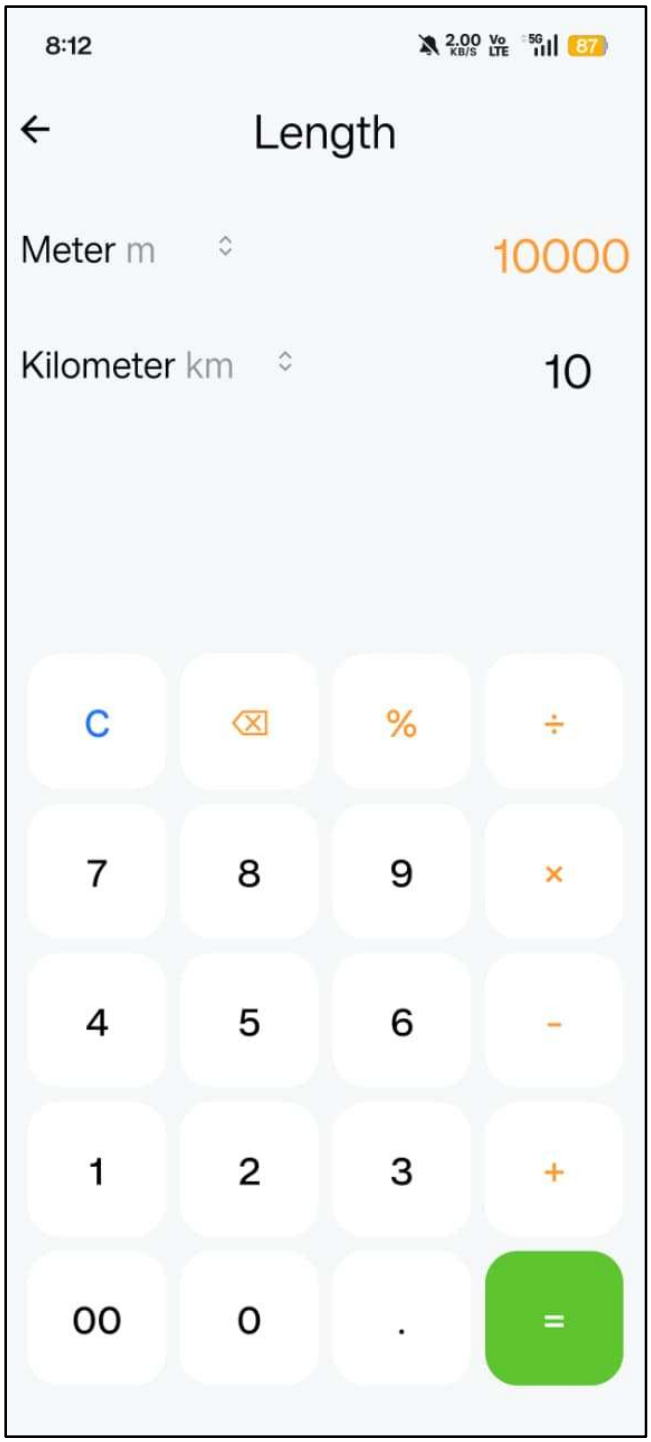


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

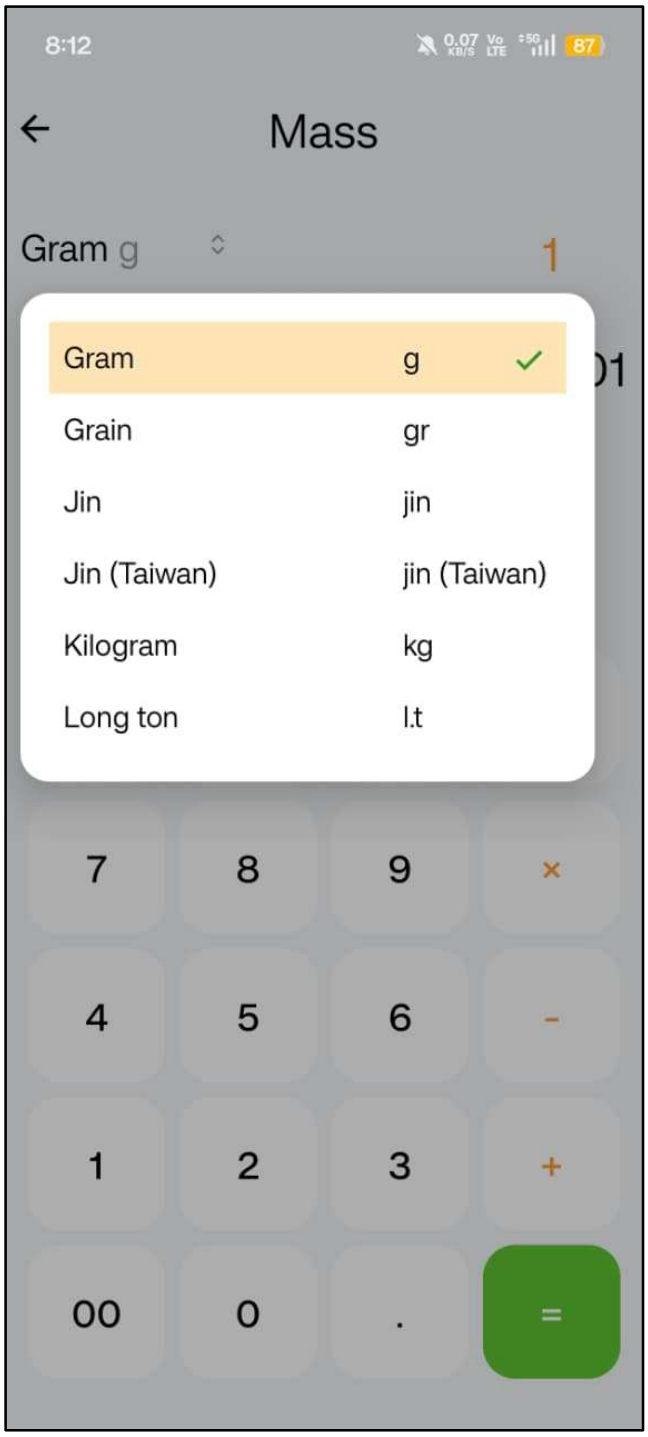


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

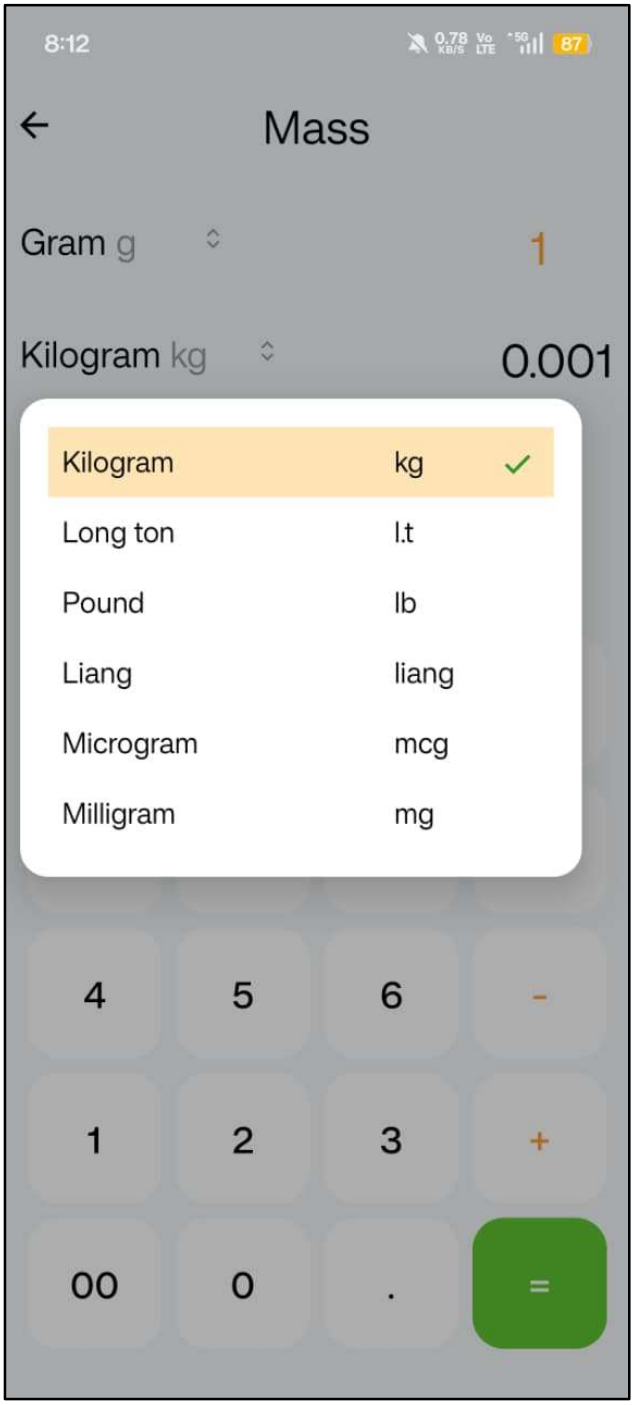


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

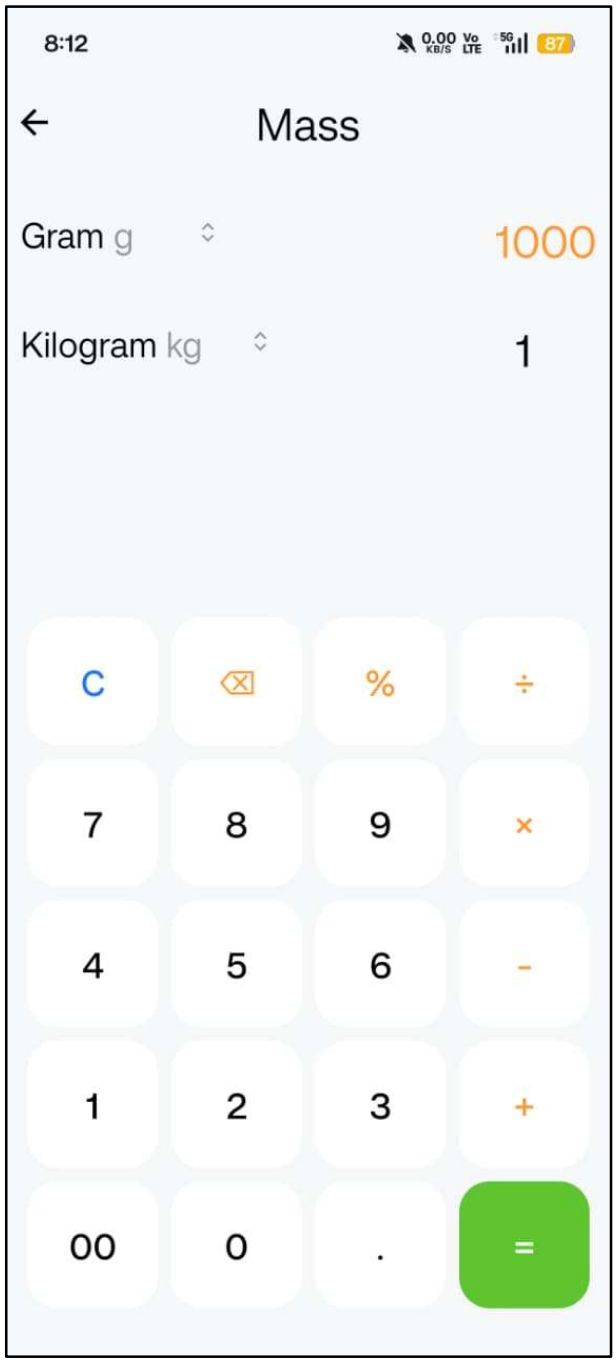


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.



Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

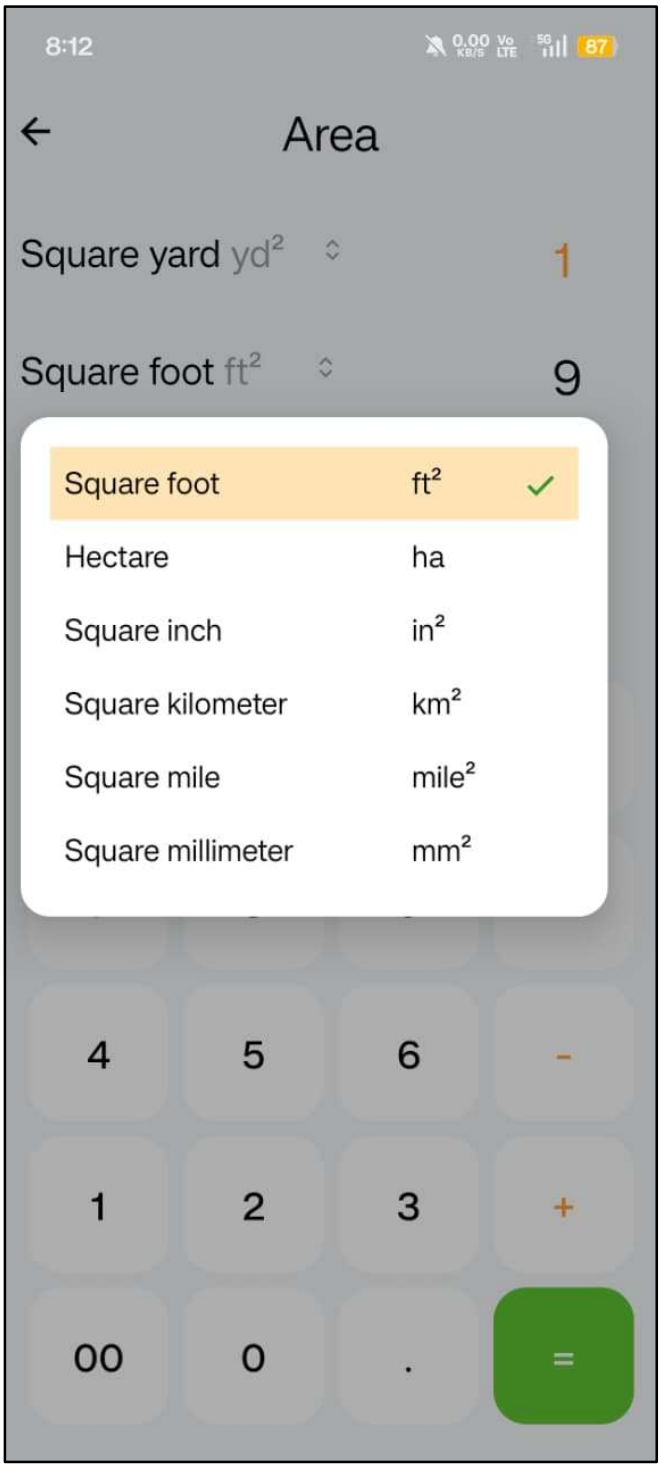
To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

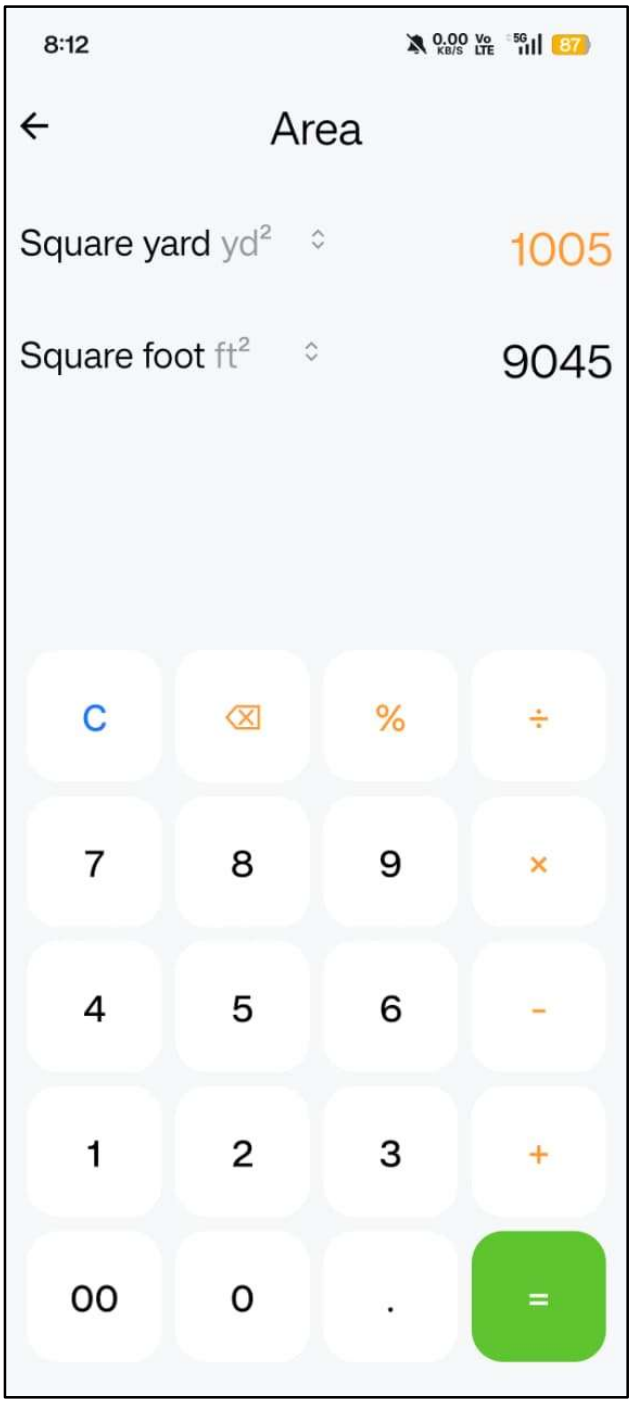


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

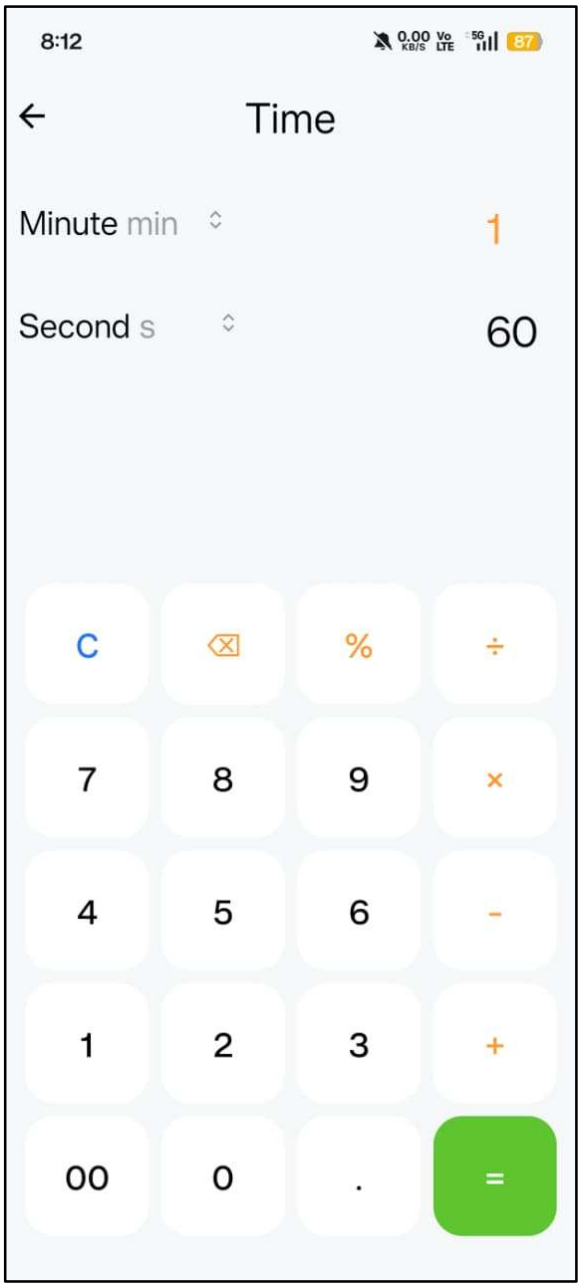


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

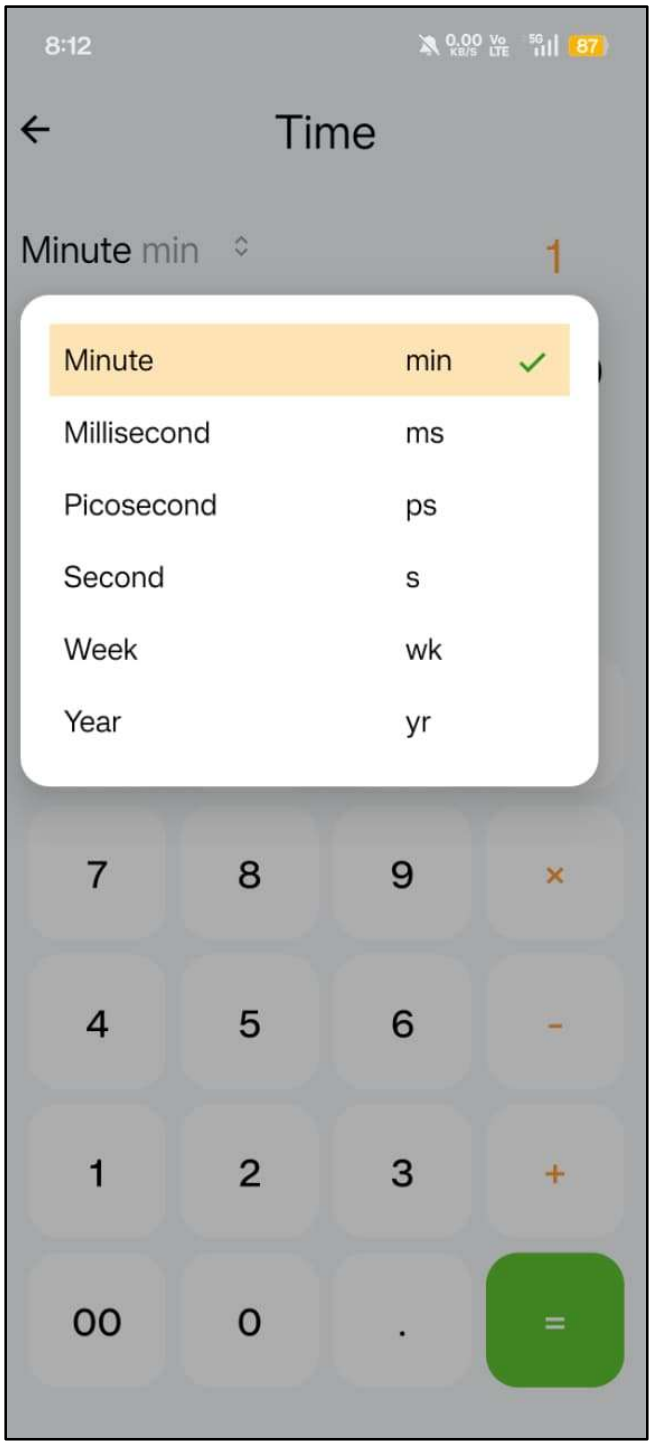


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

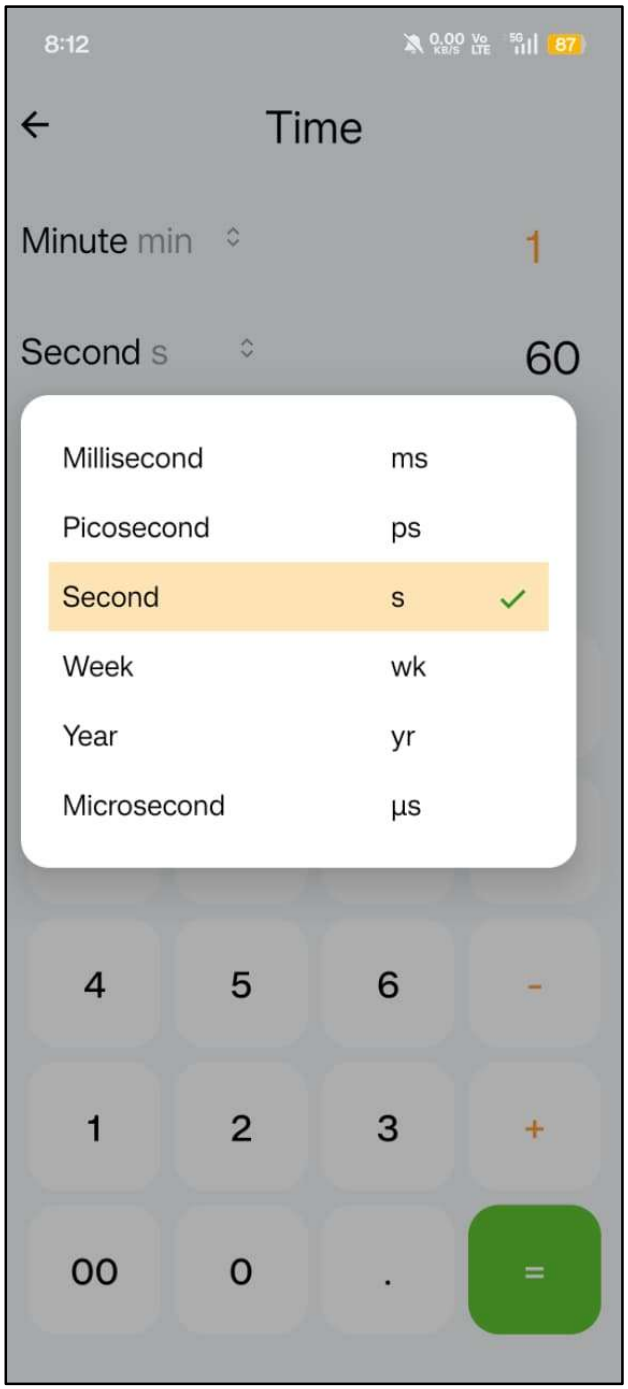


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

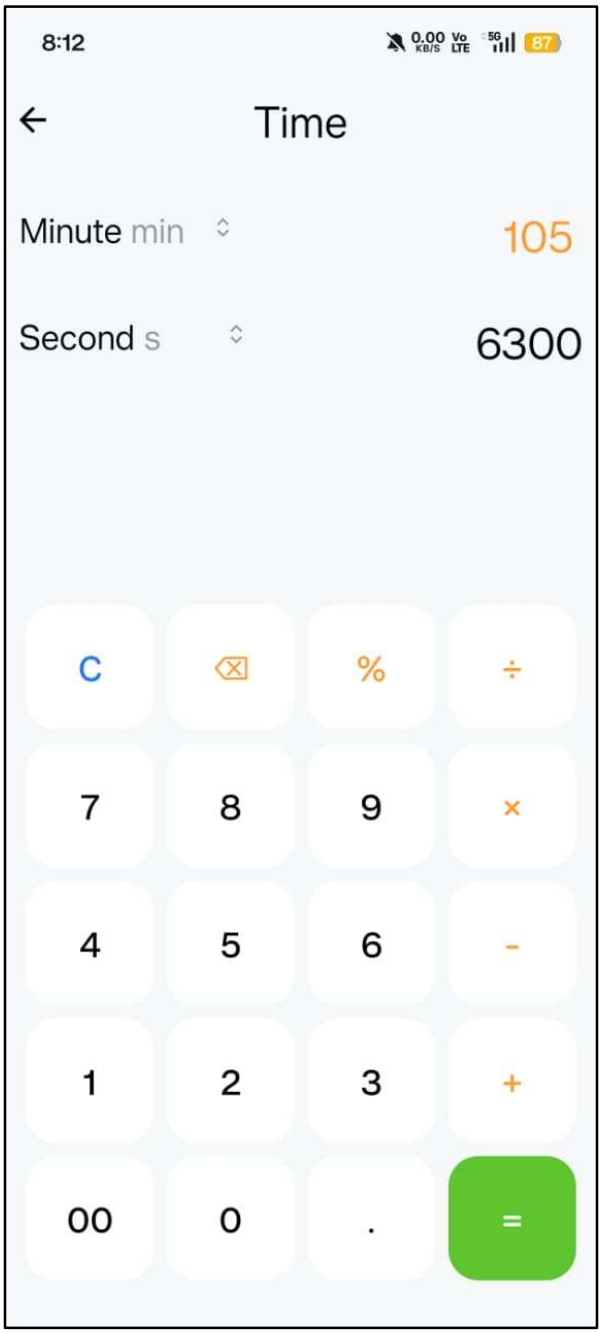


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

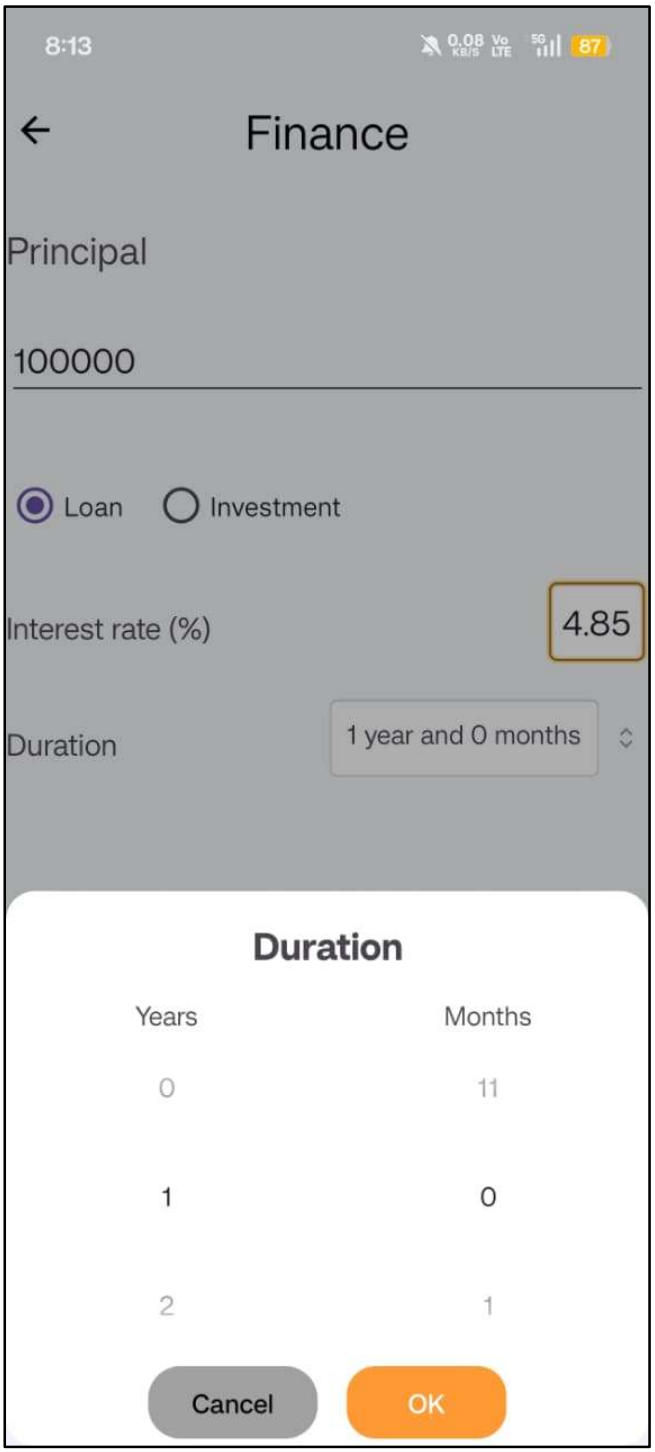


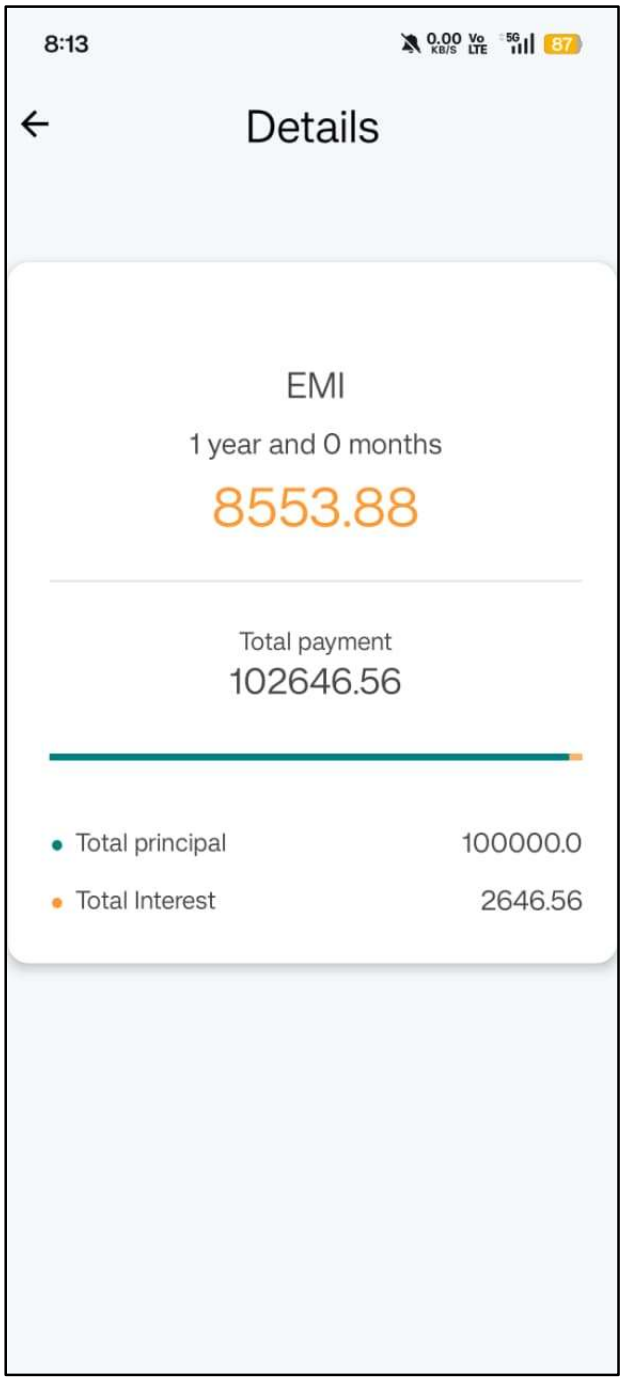
Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.



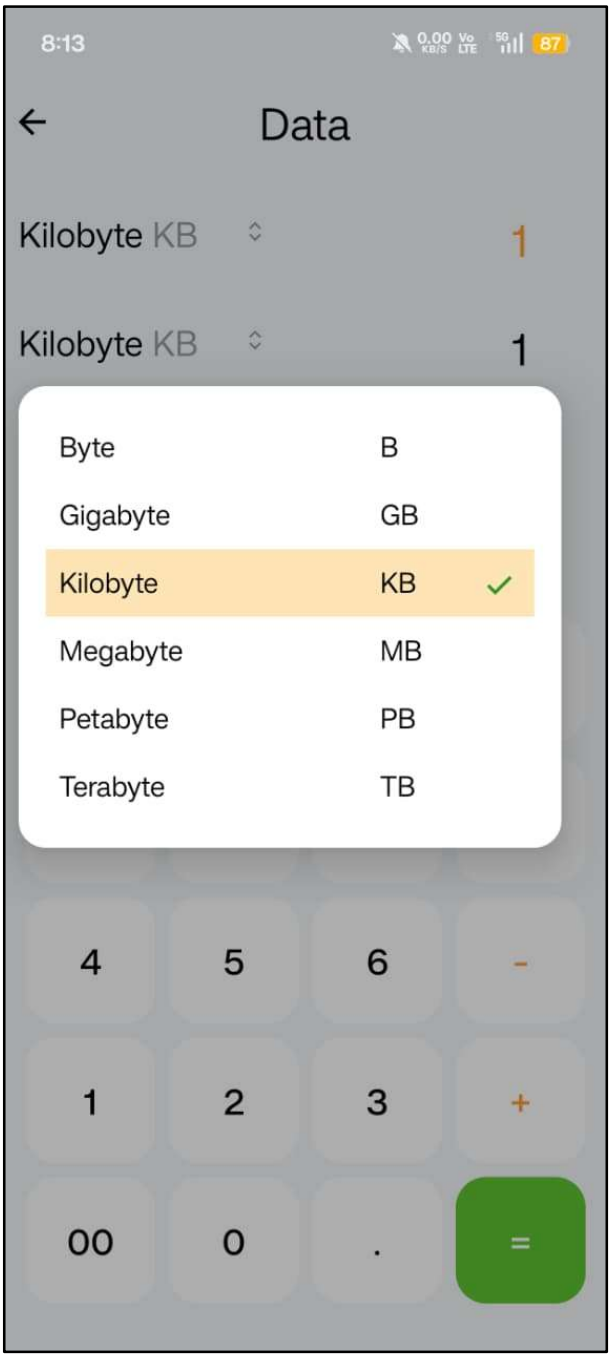

8:13
Details
EMI
1 year and 0 months
8553.88
Total payment
102646.56
Total principal
100000.0
Total Interest
2646.56

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

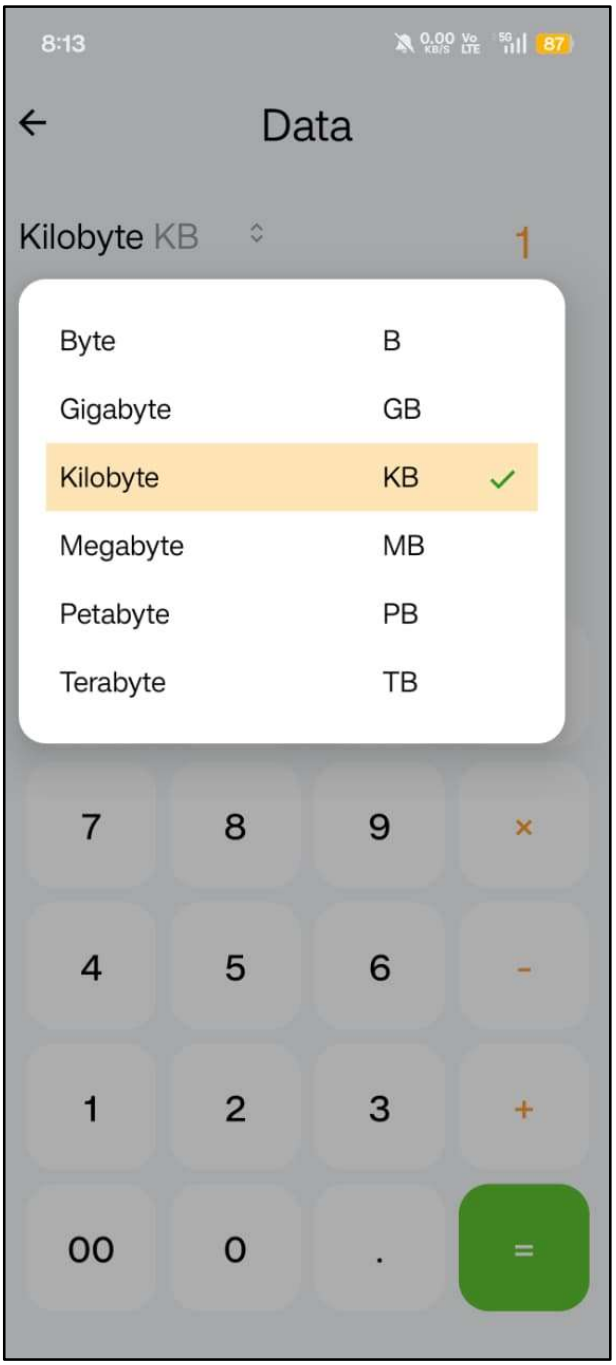


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

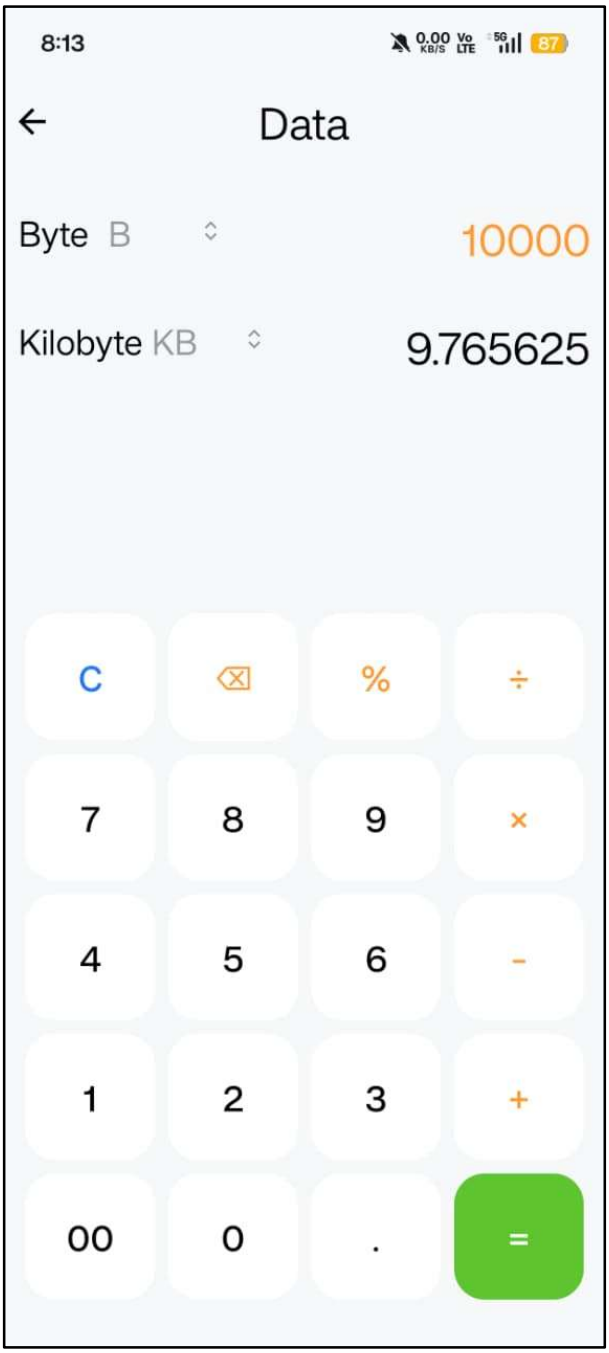


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

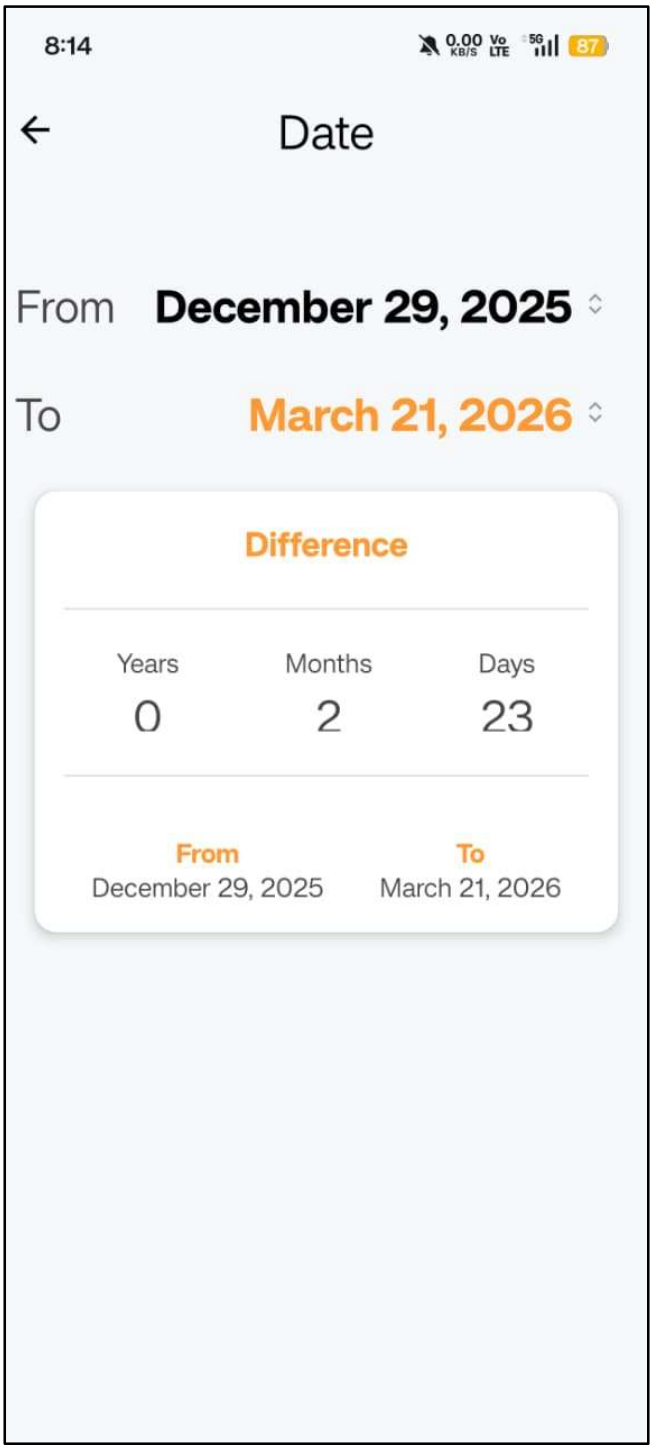


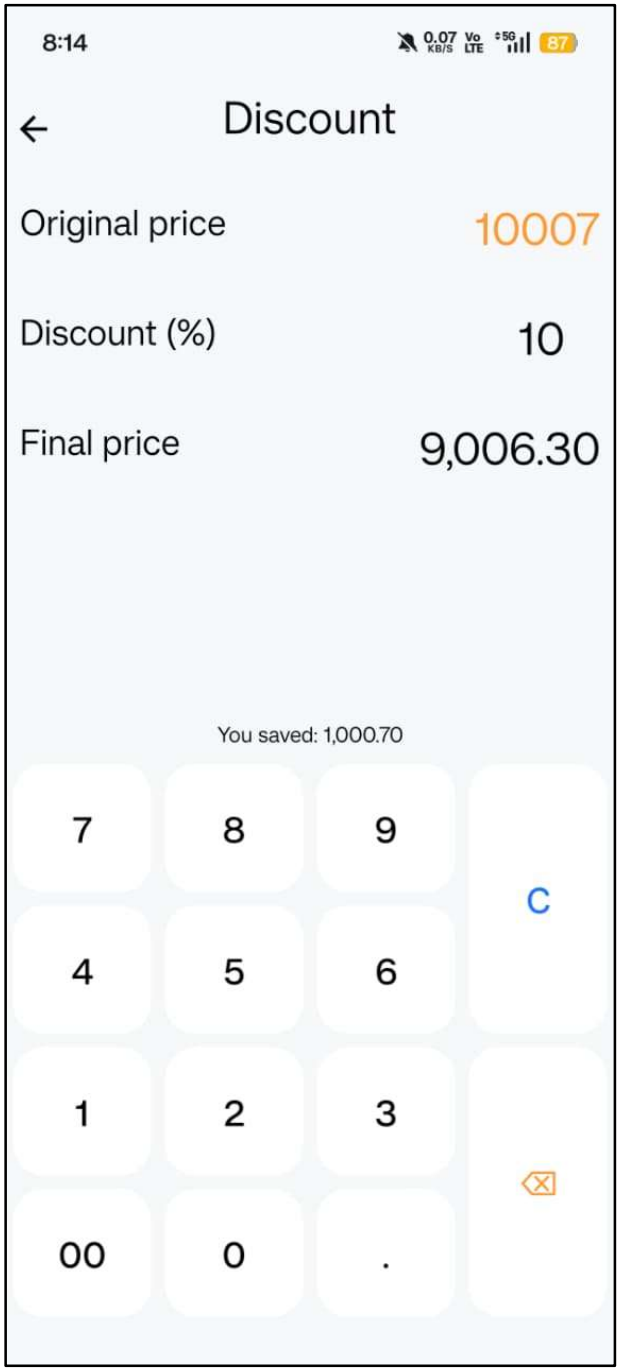
Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.



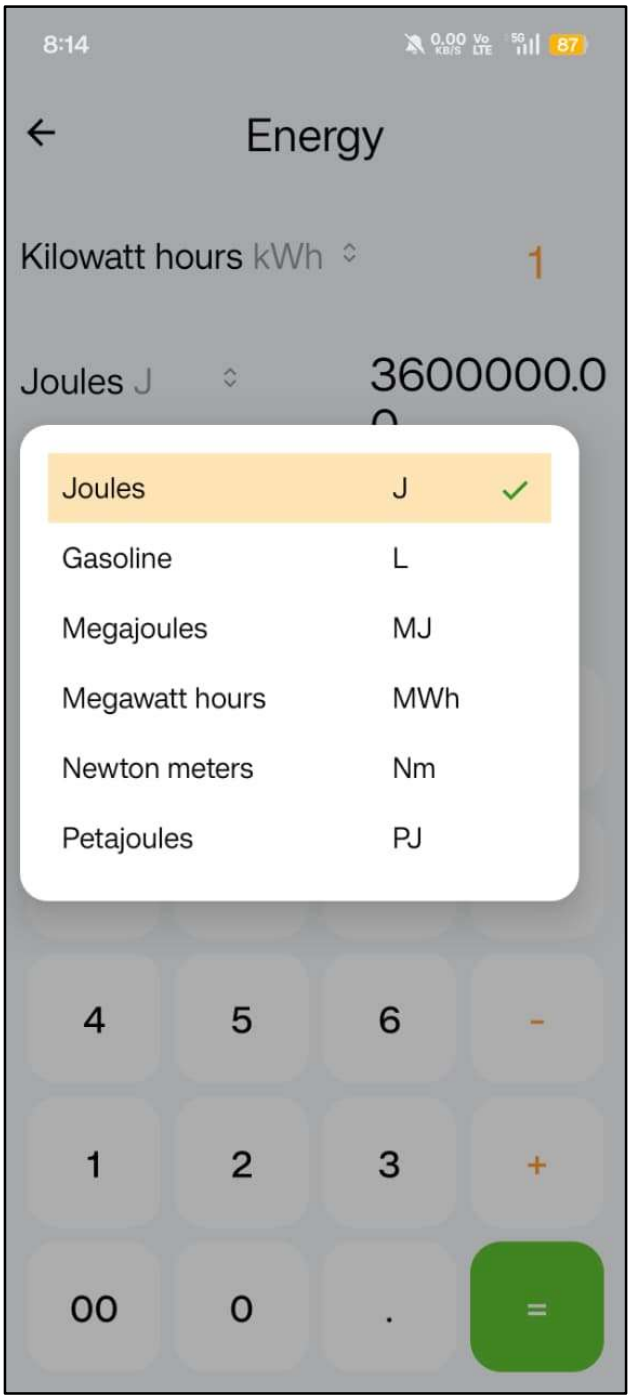

8:14
Discount
Original price
10007
Discount (%)
10
Final price
9,006.30
You saved: 1,000.70
7
8
9
C
4
5
6
1
2
3
00
0
.

Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

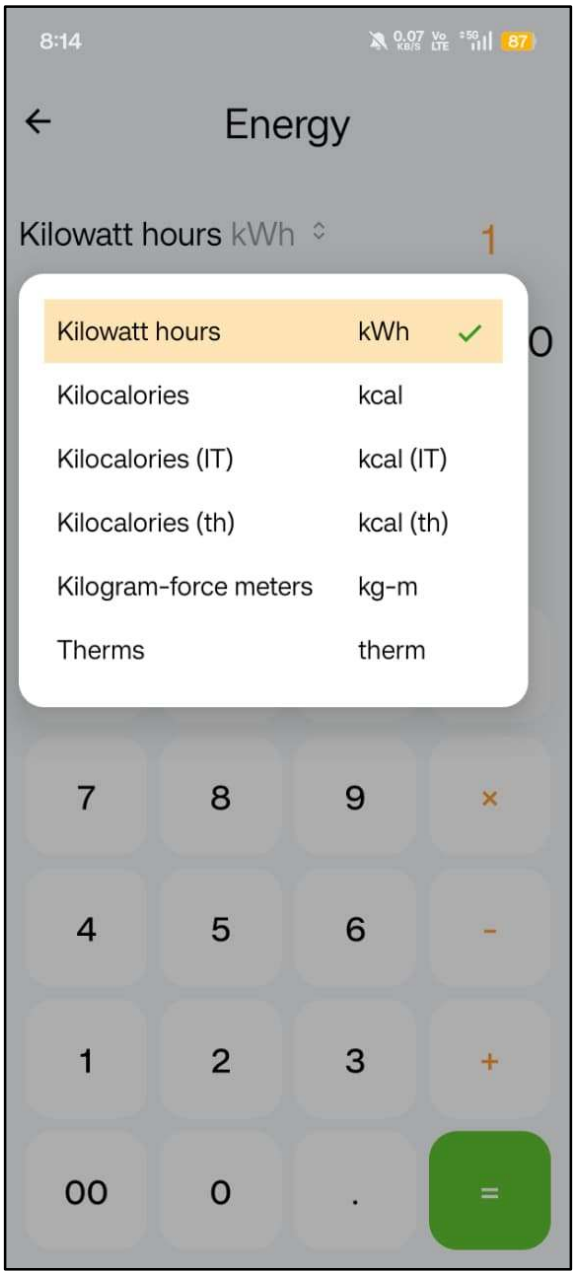


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

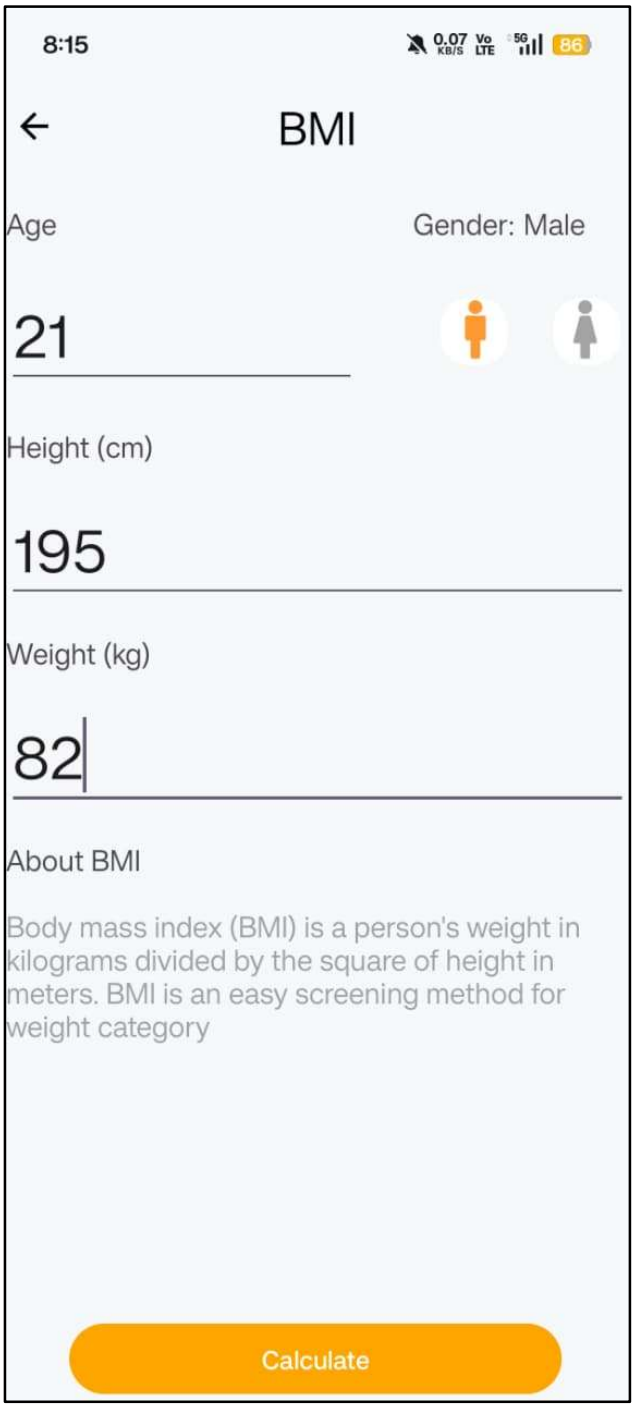


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

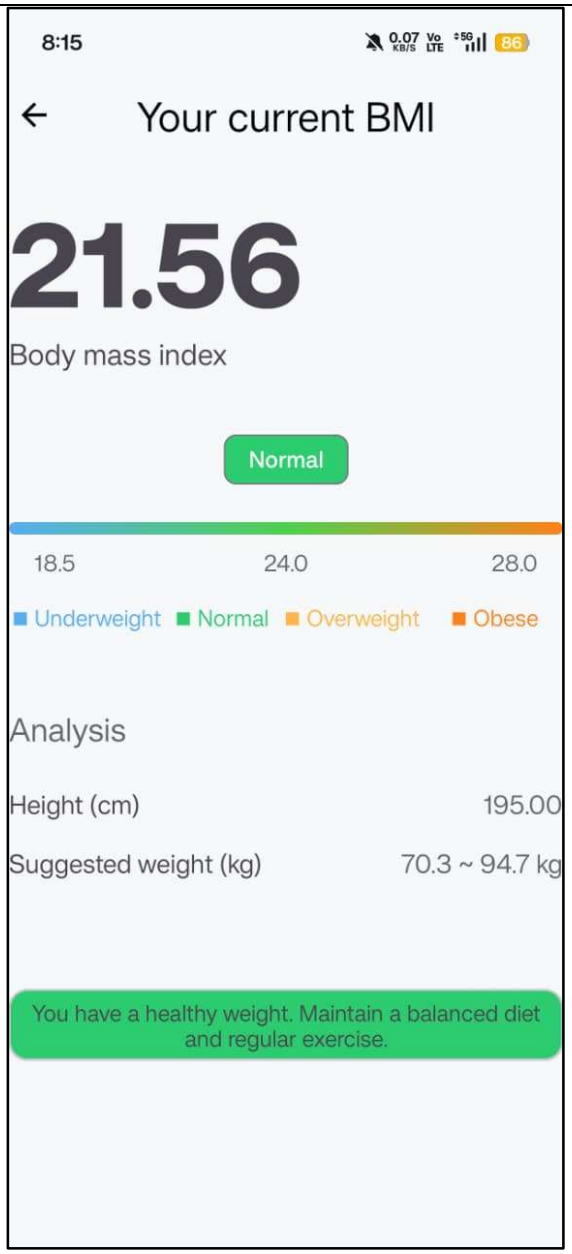


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

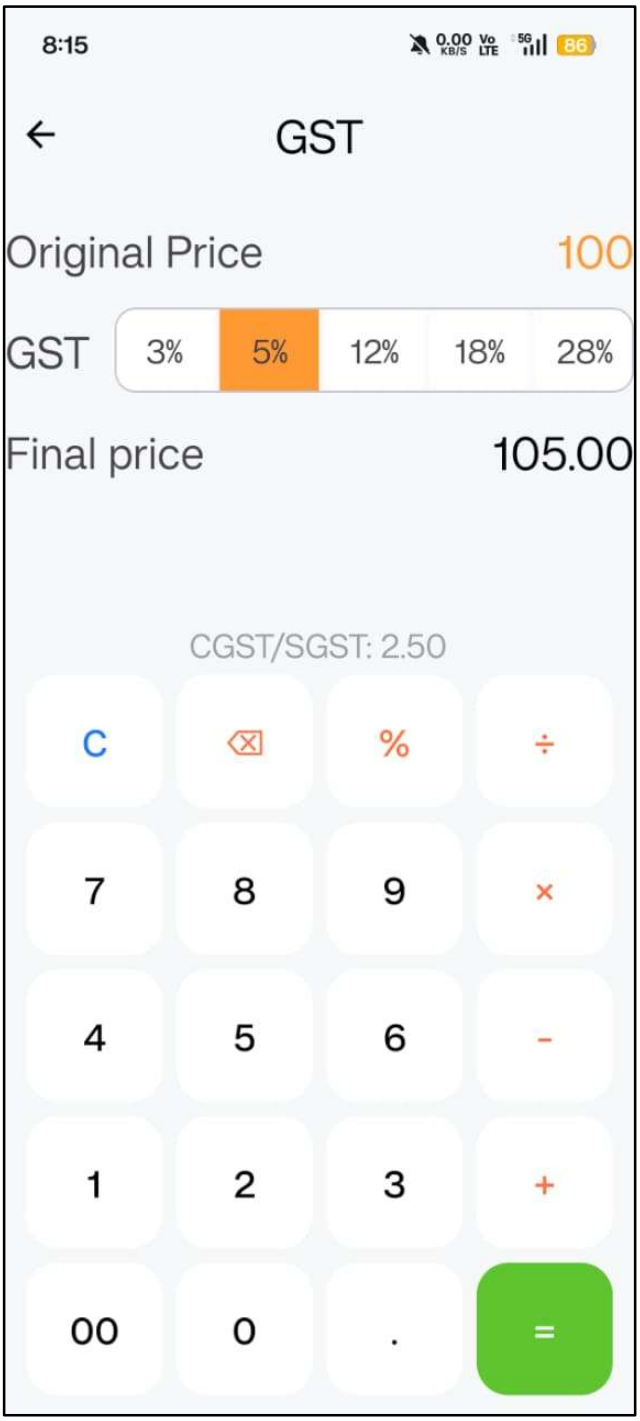


Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.

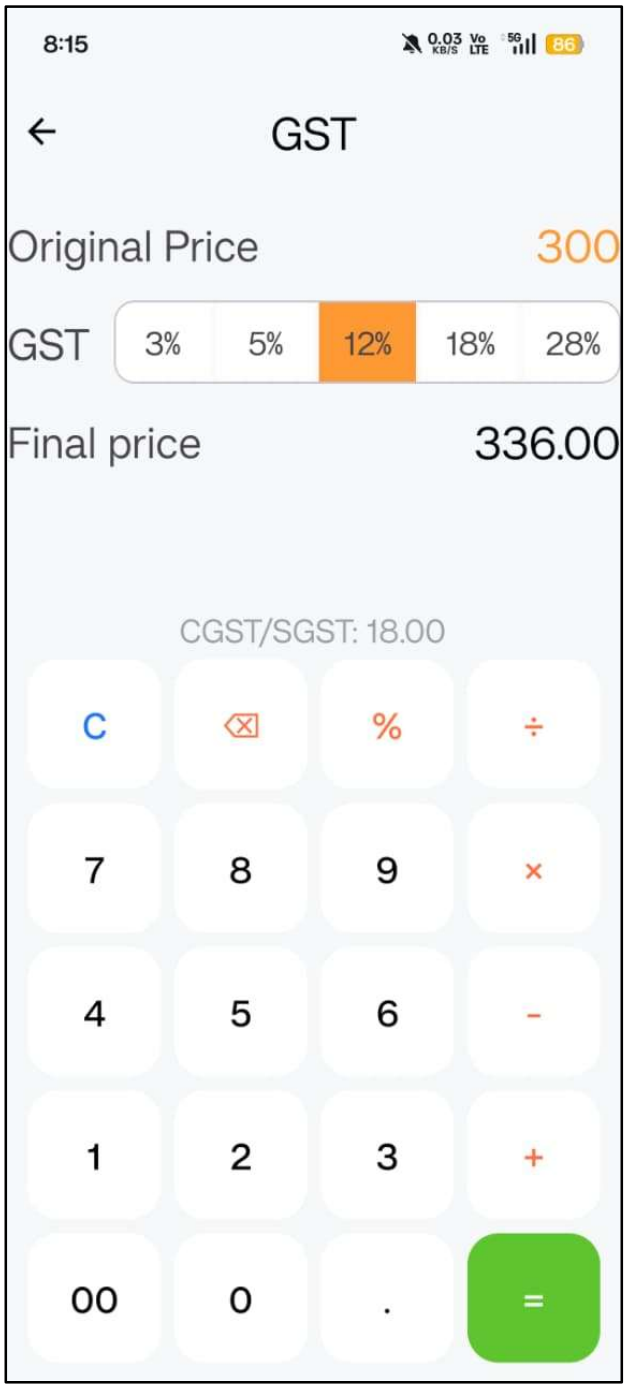


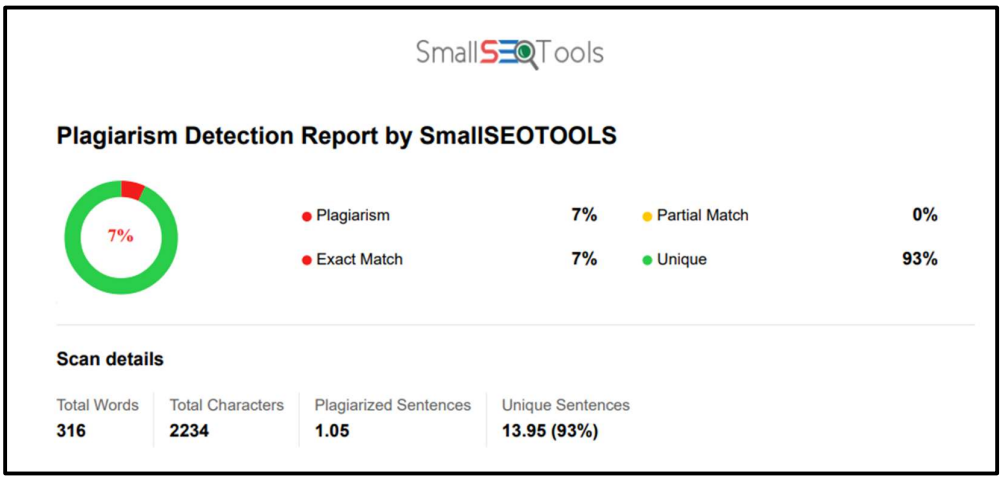
Vision of the Department

To be a preferred hub for nurturing computer engineering aspirants through innovative, ethical, value-based technical education, impactful research, and industry collaboration, to become successful professionals contributing for the betterment of society.

Mission of the Department

To nurture computer engineering professionals by offering: Dynamic learning environment that encourages problem-solving using emerging technology, · National Education Policy aligned evidence-based technical education, fostering innovation, research, and industry–institute collaboration, Holistic, ethical and sustainable development for lifelong success.



Output Analysis	The application successfully performs unit conversions with high accuracy. It provides instant results and handles multiple categories efficiently. The user interface is responsive, making it easy for users to switch between different conversion types without delay.
Link of student Github profile where lab assignment has been uploaded	https://github.com/sheikhwasimuddin
Conclusion	This experiment demonstrated how to manage UI state effectively in an Android The project demonstrates how Kotlin can be effectively used to develop a practical Android application. It simplifies everyday calculations and enhances user convenience. The app is reliable, efficient, and can be further expanded with more features in the future.
Plag Report (Similarity index < 12%)	**7%** 
Date	21-03-2026